

Machine Learning

Team 34: Project Report

Pratyush Raj (7076138) | Tejas Dhopavkar (7075870) | Parthiv Sarkar (7075665)

Summer 2025

1 Introduction

Image segmentation [1] is a task to separate the foreground from the background. Segmentation enables machines to understand images with the same level of detail as humans do - object by object, region by region. In addition, image segmentation helps in various other applications such as medical imaging, self-driving cars, augmented reality, and so on.

Image segmentation can be modeled as a supervised learning problem [2], [3], [4] or a semi-supervised learning problem [5], [6]. When image segmentation is considered as a supervised learning task, the models use the original image and the segmented image to learn the segmentation patterns. However, image segmentation with a supervised learning approach is expensive as it requires immense manual work and labeling. Semi-supervised learning overcomes this drawback by using a small amount of labeled data, scribbles, and large amount of unlabeled data. Considering both approaches, we performed multiple experiments using models like KNN and various CNN architectures, and we discuss these in the next sections.

2 Model architectures and experiments

2.1 Experiment 1: KNN with Logistic Regression

We first applied the k-Nearest Neighbors (KNN) model [7] to perform image segmentation based on local contextual information. In this method, we used KNN combined with logistic regression refinement and optional contour based cleanup. The method takes as input an RGB image and a corresponding scribble mask, where scribbles represent weak annotations of background and foreground regions, while the majority of the image remains unmarked.

First, the KNN algorithm is applied by using pixel coordinates as input features, and labeled pixels are employed to train the classifier. This results in an initial propagation of scribbles across the full image, ensuring every pixel receives a label.

In the second step, the RGB values are smoothed with a Gaussian filter to reduce noise, and then flattened into feature vectors. To further improve accuracy, the predictions from KNN are used as pseudo-labels and combined with the original scribble labels. Logistic regression is then applied with sample weights that account for prediction confidence, allowing the model to learn a linear decision boundary in feature space. By integrating KNN with logistic regression and utilizing the prediction confidence, this model effectively filters out noisy or uncertain labels, resulting in better generalization performance as seen in figure 2.

For post-processing, we employ contour-based filtering to remove minor noisy regions, leveraging OpenCV's contour detection functionality. This ensures that the segmentation result is spatially coherent and less fragmented. However, KNN leads to noisy blobs. In order to avoid relying solely on distance metrics for similarity and to incorporate both low-level (e.g., edges, textures) and high-level (e.g., object shapes) image features, we explored the use of convolutional neural networks (CNNs).

2.2 Experiment 2: CNN

In this approach, we used the convolutional neural networks proposed in [8] to segment the images. Earlier, image segmentation was performed using a sliding window approach, where cropped regions of the image were input to a CNN that predicted the label of the central pixel. This method was inefficient because it required independent processing of each crop and failed to utilize features shared between overlapping crops.

To overcome the inefficiencies of the sliding window approach, [2] introduced a method in which the full image is passed through a CNN architecture. This was fast and efficient as the entire image had to be processed only once and it was now possible to assign a category label to each pixel at once. We used this approach in this experiment.

The CNN consists of 5 hidden layers with each filter having a spatial dimension of 3×3 as depicted in figure 1. We also apply a padding and a stride of 1 to ensure that the intermediate feature maps share the same spatial dimensions as the input image. The size of the padding and stride were determined by the equations given in equations (1) and (2). Each layer of the CNN has 40% of the neurons dropped to add some extra regularization, as the neurons are forced to work independently and avoid over-fitting. Also, each layer is inter-spaced with a ReLU [9] non-linear activation function, while the final layer uses Sigmoid activation to classify the pixels as belonging to the foreground or the background.

As seen from the evaluation of the output in figure 3, simply using a CNN with no scribbles does not yield promising results. Also, downsampling is not used in this architecture, hence the training process is quite slow. To overcome these shortcomings, we perform experiment 3.

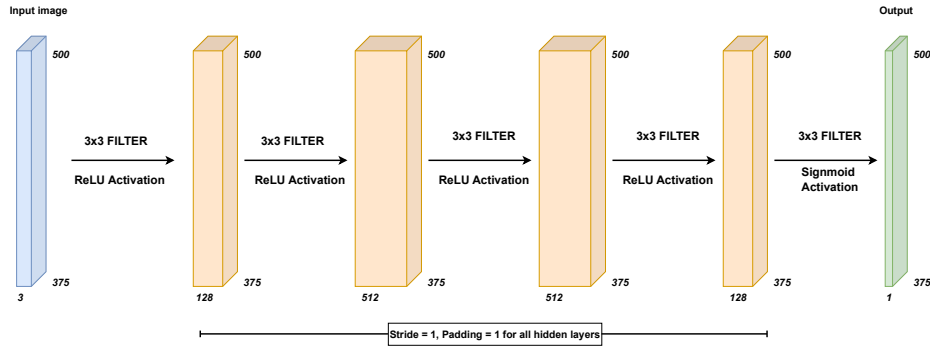


Figure 1: Sample image with scaling.

$$H' = \frac{N - F + 2P}{S} + 1 \quad (1)$$

$$W' = \frac{N - F + 2P}{S} + 1 \quad (2)$$

2.3 Experiment 3: DeepLabV3+

The dataset includes 375×500 RGB images $\mathbf{b} \in \mathbb{R}^{n_x \times n_y \times 3}$, scribbles $\mathbf{c} \in \{0, 1, \emptyset\}^{n_x \times n_y}$, and ground-truth masks $\mathbf{g} \in \{0, 1\}^{n_x \times n_y}$. Scribbles cover $< 5\%$ of pixels, requiring label propagation. We compute distance maps using equations (3) and (4) with $\sigma = 50$, and normalize RGB by 255 to form a 5-channel input $\mathbf{x}$. Data is split 80% and 20% for training and validation respectively, using PyTorch's *random-split*.

$$d_{fg}(i, j) = \exp\left(-\frac{[EDT(1 - m_{fg})](i, j)^2}{2\sigma^2}\right) \quad (3)$$

$$d_{bg}(i, j) = \exp\left(-\frac{[EDT(1 - m_{bg})](i, j)^2}{2\sigma^2}\right) \quad (4)$$

We adapt DeepLabV3+ [10] with a ResNet-like encoder [11]. The encoder uses a 7×7 convolution (stride 2, 64 channels), BN, ReLU, and max grouping, followed by four layers of BasicBlocks with [2,2,2,2] blocks, channels [64,128,256,512], strides [1,2,2,1], and dilations [1,1,1,2]. ASPP applies 1×1 and 3×3 atrous convolutions (rates [12,24,36]), global pooling, and projects to 256 channels. The decoder upsamples, concatenates low-level features (48 channels), and outputs a 1-channel logit map.

$$\text{BasicBlock}(x) = \text{ReLU}(\text{BN}(\text{Conv}_{3 \times 3}(\text{ReLU}(\text{BN}(\text{Conv}_{3 \times 3}(x)))))) + x' \quad (5)$$

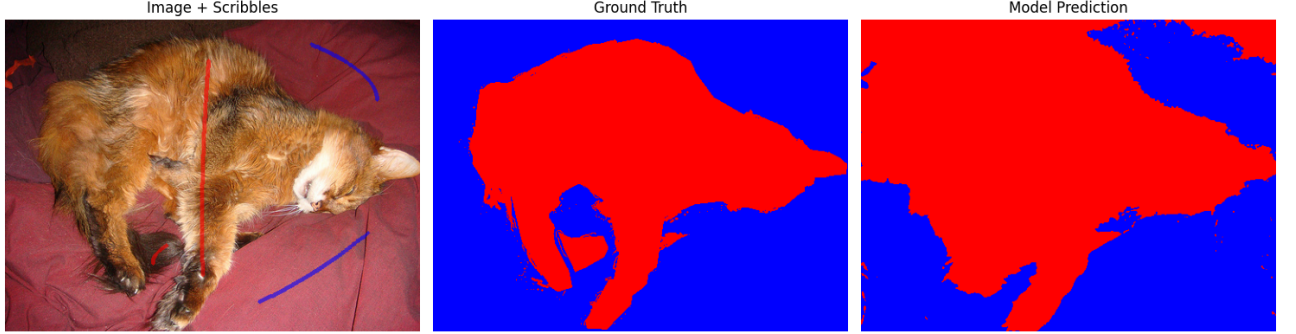


Figure 2: Sample prediction of the KNN + Logistic Regression Model.

3 Evaluation

We evaluated our model only on the training dataset that was provided to us. No other dataset was used.

Table 1: Training Setup.

Component	Experiment 1	Experiment 2	Experiment 3
Model	KNN	CNN	DeepLabV3+ with ResNet Encoder
Epochs	-	25	100
Loss Function	-	Binary Cross Entropy	Binary Cross Entropy with Sigmoid
Optimizer	-	Adam	Adam
Learning Rate	-	1×10^{-4}	1×10^{-3}

Table 2: Evaluation Metrics on Training Dataset for Experiment 3 in section 2.3.

Metric	Epoch 1	Epoch 10	Epoch 20	Epoch 40	Epoch 80
Loss	0.6923	0.1234	0.0987	0.0765	0.0685
mIoU	0.5034	0.7125	0.7456	0.7678	0.7646
Object IoU	0.3215	0.5456	0.5987	0.6321	0.6273

The measure used for the evaluation is IoU (Intersection over Union). It calculates the measure of overlap between the ground truth and the prediction. We started with a basic KNN with Logistic Regression based classifier that takes the local context into consideration and takes into account nearby samples in feature space. It worked reasonably well when foreground

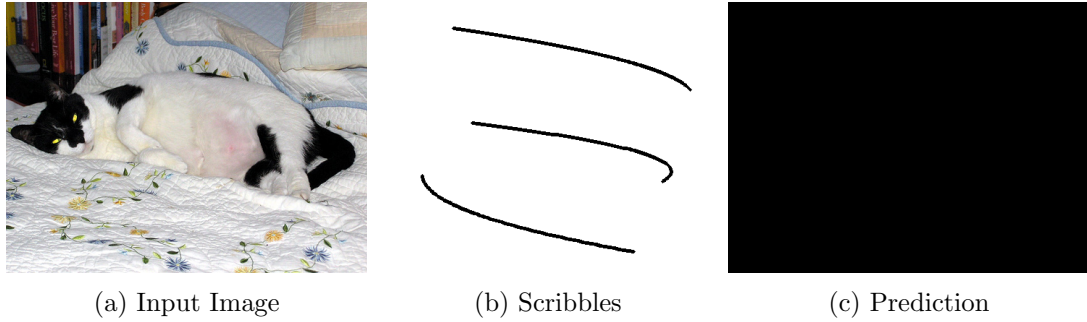


Figure 3: Sample prediction of the CNN Model.

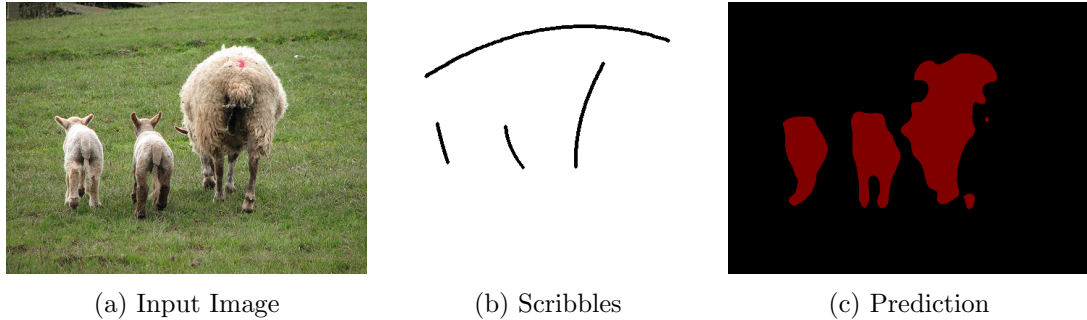


Figure 4: Sample prediction of the DeepLabV3+ Model.

and background are clearly separated and also in cases where the scribble only had foreground or background information.

The main problem with the KNN based model was it produced noisy masks with irregular boundaries since it only propagates scribbles locally without learning higher-level patterns. To ensure that all types of features like edges, textures and object shapes are captured, we experimented with CNN. However, due to lack of downsampling and not using scribbles, the output was not good. This was then resolved by DeepLabV3+ with ResNet encoder model. It produced smooth, well-structured masks that align closely with true object boundaries. Skip connections also helped in boundary refinement, avoiding noisy masks seen in KNN. ASPP in DeepLabV3 captures large receptive fields, allowing the network to use long-range dependencies.

4 Conclusion

In this project, we implemented three approaches to semi-supervised image segmentation: KNN with logistic regression, a basic CNN, and DeepLabV3+ with a ResNet encoder. The KNN-based method worked well for simple cases where foreground and background were clearly separated but produced noisy and irregular masks due to its inability to capture high-level image features. The CNN was implemented to improve over KNN by learning spatial patterns across the entire image, but the lack of downsampling and reliance on a shallow architecture limited its efficiency and segmentation accuracy.

DeepLabV3+ achieved the best results, as shown in the evaluation tables, with smooth, coherent masks and higher IoU scores, demonstrating its ability to capture both global semantic context and fine boundary details through ASPP and skip connections. While this model required greater computational resources and longer training, it significantly outperformed the simpler methods. Overall, the experiments highlight a clear progression: classical methods like KNN are useful as baselines, but modern architectures such as DeepLabV3+ are essential for achieving robust, high-quality segmentation in complex real-world scenarios.

References

- [1] Shervin Minaee, Yuri Boykov, Fatih Porikli, Antonio Plaza, Nasser Kehtarnavaz, and Demetri Terzopoulos. Image segmentation using deep learning: A survey, 2020. [1](#)
- [2] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation, 2015. [1](#), [2](#)
- [3] Wei Liu, Andrew Rabinovich, and Alexander C. Berg. Parsenet: Looking wider to see better, 2015. [1](#)
- [4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. [1](#)
- [5] Adrian Peláez-Vegas, Pablo Mesejo, and Julián Luengo. A survey on semi-supervised semantic segmentation, 2023. [1](#)
- [6] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014. [1](#)
- [7] T. Cover and P. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967. [1](#)
- [8] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. [2](#)
- [9] Abien Fred Agarap. Deep learning using rectified linear units (relu), 2019. [2](#)
- [10] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation, 2018. [3](#)
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. [3](#)